

Rapport technique - traduction-audio

Rapport technique

Traduction-audio : plateforme LLMOps de traduction audio temps réel

Table des matières

1. Résumé
2. Le besoin auquel je réponds
3. Ce que j'ai construit
4. Le paysage technique existant
5. Ma méthodologie
6. Architecture d'ensemble
7. Les choix techniques que j'ai faits
8. Ingénierie des données et évaluation
9. Sécurité
10. Observabilité
11. Déploiement et intégration continue
12. Résultats et chiffres
13. Difficultés rencontrées
14. Perspectives
15. Conclusion
16. Annexes techniques

Rapport technique

Traduction-audio : plateforme LLMOps de traduction audio temps réel

Auteur : Tetyana Tarasenko Mentor : Sébastien, DataScientest Date de
soutenance : 3 septembre 2026 Repository : [github.com/TatianaT13/
translate-audio-NLP-Ai](https://github.com/TatianaT13/translate-audio-NLP-Ai) Production : <https://traduction-audio.fr>

Table des matières

1. Résumé
2. Le besoin auquel je réponds

3. Ce que j'ai construit
 4. Le paysage technique existant
 5. Ma méthodologie
 6. Architecture d'ensemble
 7. Les choix techniques que j'ai faits
 8. Ingénierie des données et évaluation
 9. Sécurité
 10. Observabilité
 11. Déploiement et intégration continue
 12. Résultats et chiffres
 13. Difficultés rencontrées
 14. Perspectives
 15. Conclusion
 16. Annexes techniques
-

1. Résumé

Traduction-audio est une plateforme de traduction audio temps réel que j'ai conçue, développée et mise en production dans le cadre de mon projet de fin d'études. L'utilisateur envoie un audio en français, la plateforme le transcrit, le traduit dans la langue de son choix (anglais, espagnol, allemand, italien ou ukrainien) et lui renvoie un fichier audio synthétisé dans cette langue.

Le point de départ du projet, ce sont les flash-infos autoroutières de la radio 107.7. Ces alertes trafic sont diffusées uniquement en français, alors que les autoroutes françaises accueillent chaque année plusieurs millions d'usagers étrangers. Un chauffeur ukrainien, un touriste allemand ou un transporteur espagnol ne comprend pas quand la radio annonce un accident, un bouchon ou une déviation.

Techniquement, j'ai choisi une approche LLMOps plutôt que MLOps classique. Je ne réentraîne pas de modèles, je m'appuie sur des modèles pré-entraînés existants (Whisper pour la reconnaissance vocale, un LLM pour la traduction, un TTS pour la synthèse) que j'orchestre dans une chaîne fiable, observable et sécurisée. L'ensemble tourne dans 14 conteneurs Docker déployés sur un VPS Hetzner, avec du HTTPS, une authentification JWT, du monitoring Prometheus et Grafana, du tracing MLflow et Langfuse, un batch nocturne d'évaluation orchestré par Airflow, et une intégration continue via GitHub Actions.

Trois fonctionnalités principales sont livrées : la traduction à la demande depuis un fichier ou le micro, un enregistreur de réunion avec compte-rendu automatique, et une traduction simultanée en direct via WebRTC connecté à l'API OpenAI Realtime. Ces deux dernières features n'étaient pas prévues au départ. Elles ont émergé pendant le développement et ouvrent des débouchés produits intéressants.

2. Le besoin auquel je réponds

Autoroute Info diffuse ses flash-infos sur la fréquence 107.7 en continu, 24 heures sur 24. Les zones couvertes vont de l'Île-de-France à la Bourgogne, en passant par le nord, le sud, l'est et l'ouest. Le contenu est structuré : accidents, bouchons, travaux, animaux sur la chaussée, fermetures de voies, événements météo. Toutes ces informations sont critiques pour la sécurité, mais elles ne sont émises qu'en français.

En pratique, cela crée un déséquilibre. Un usager français peut anticiper une déviation ou ralentir avant un bouchon, tandis qu'un usager étranger conduit sans cette information. Les conséquences sont concrètes : accidents secondaires liés à un ralentissement non anticipé, freinages tardifs sur bouchon, sentiment général d'exclusion des services publics de sécurité routière.

Le défi technique n'est pas anodin. Il ne s'agit pas de traduire des documents statiques mais un flux audio, souvent bruité (moteur, radio, vitres ouvertes), avec des noms propres, des numéros de sorties et un vocabulaire spécifique. Il faut aussi que la chaîne complète soit rapide, autour de deux ou trois secondes de latence, sinon l'information arrive après le bouchon.

Enfin, il faut que cette chaîne soit fiable. Un LLM peut halluciner, un provider cloud peut tomber, un audio peut contenir une tentative d'injection de prompt. Toutes ces préoccupations de production sont au cœur de mon approche LLMOps.

3. Ce que j'ai construit

La plateforme propose trois modes d'utilisation qui répondent à trois cas d'usage distincts.

Le premier mode est la traduction à la demande. L'utilisateur dépose un fichier MP3 ou WAV sur l'interface, ou bien il enregistre directement au micro depuis son navigateur. Il choisit la langue cible, valide, et reçoit après quelques secondes la transcription en français, la traduction en langue cible, et un fichier audio synthétisé qu'il peut écouter ou télécharger. C'est le cas d'usage historique, celui qui répond directement au besoin des chauffeurs étrangers.

Le deuxième mode est un enregistreur de réunion, qui a émergé pendant le développement. L'utilisateur lance un enregistrement micro long, la plateforme découpe le flux en morceaux de trente secondes, les transcrit à la volée en affichant le texte au fur et à mesure, puis génère à la fin un compte-rendu structuré via un LLM. L'utilisateur choisit entre trois styles de résumé : synthétique pour les décisions, détaillé pour un procès-verbal, ou uniquement les actions à faire. Cette fonctionnalité constitue une vraie piste de monétisation SaaS, sur un marché déjà porteur (Otter, Fireflies, Grain).

Le troisième mode est la traduction simultanée en direct, via WebRTC. Ici le navigateur se connecte directement à l'API OpenAI Realtime, en récupérant au préalable un token éphémère auprès de mon serveur (afin de ne jamais

exposer ma vraie clé OpenAI côté client). L'utilisateur parle, la traduction sort dans son casque avec environ 500 millisecondes de latence. Cette feature transforme la démonstration : le jury peut littéralement parler en français et entendre l'anglais en temps réel.

En parallèle de ces trois modes, un service backend nommé Watcher tourne en continu. Il interroge périodiquement le flux radio 107.7, transcrit les nouveaux flash-infos avec un Whisper embarqué, extrait les événements structurés (zone, sévérité, type), les traduit dans les cinq langues cibles, et les pousse en Server-Sent Events vers un dashboard administrateur. Cela permet à un opérateur de voir en direct ce qui se passe sur le réseau autoroutier français, sans avoir à écouter la radio.

Enfin, le dashboard administrateur est un centre de contrôle MLOps complet. On y voit les métriques infrastructure, les expériences MLflow, les DAGs Airflow, la liste des utilisateurs, les traces LLM Langfuse et les coûts cumulés. L'accès est protégé par un rôle `is_admin` sur le compte utilisateur.

4. Le paysage technique existant

Avant de commencer, j'ai regardé ce qui existe. Plusieurs solutions occupent le marché de la traduction et de la reconnaissance vocale, mais aucune ne couvre exactement mon besoin.

Google Translate API est la référence pour la traduction texte, mais c'est une boîte noire. Impossible d'ajuster le prompt, impossible de tracer une requête, impossible de choisir un modèle plutôt qu'un autre. DeepL propose une meilleure qualité de traduction, notamment en allemand, mais son API n'inclut ni STT ni TTS, ce qui m'obligerait à composer une chaîne complète moi-même.

AssemblyAI fait de l'excellent STT en streaming, avec des latences très faibles, mais coûte 0,015 dollar par minute audio, ce qui devient prohibitif à volume. L'API Whisper d'OpenAI est simple et précise, mais elle facture 0,006 dollar par minute, sans compter que je n'ai aucun contrôle sur la version du modèle utilisée en coulisses. Azure Speech Translation propose une pipeline speech-to-speech complète, mais son SDK est lourd, orienté entreprise, et l'intégration demande beaucoup de configuration.

J'ai donc choisi de construire ma propre pipeline en assemblant les briques les plus adaptées à chaque étape. Whisper large-v3 en local pour la transcription, un LLM cloud via LiteLLM pour la traduction (avec fallback multi-provider), Voxtral et MMS-TTS pour la synthèse vocale selon la langue cible. Cette approche me donne la maîtrise complète du coût, de la latence, du prompt, et des versions de modèles.

5. Ma méthodologie

Mon approche s'inscrit dans la démarche LLMOps, qui diffère du MLOps traditionnel sur un point essentiel. En MLOps classique, l'enjeu principal est d'entraîner et de re-entraîner des modèles custom. Ici, je n'entraîne rien. Les modèles sont pré-entraînés et téléchargés. Ce qui me demande de la rigueur, c'est leur orchestration, leur observation et leur mise à jour, dans une chaîne qui doit rester stable en production.

J'ai découpé le projet en quatre phases.

La première phase, de mars à avril 2026, a été consacrée à l'ingénierie des prompts et à la sélection des modèles. J'ai construit un dataset de référence (« golden ») composé de trente flash-infos réels, transcrits et traduits à la main dans les cinq langues cibles. J'ai ensuite testé douze configurations différentes, croisant deux tailles de modèle Whisper, deux tailles de LLM et trois versions de prompt. Chaque configuration a été évaluée sur les cinq audios du golden, soit soixante runs au total. Cette phase m'a donné un choix objectif de la combinaison gagnante.

La deuxième phase, d'avril à mai, a consisté à découper le système en microservices FastAPI conteneurisés, à mettre en place MLflow comme registre de modèles et Langfuse comme registre de prompts et de traces.

La troisième phase, en mai et juin, a été l'assemblage. J'ai construit le pipeline central en LangChain LCEL, ajouté la gateway avec authentification JWT, et déployé le tout sur mon VPS Hetzner en HTTPS.

La quatrième phase, en juillet et août, a été consacrée au monitoring et à l'évaluation batch. J'ai ajouté Prometheus et Grafana pour les métriques infrastructure, activé le tracing MLflow et Langfuse dans le pipeline, et créé deux DAGs Airflow (un nocturne pour évaluer la qualité sur le golden, un hebdomadaire pour détecter les dérives).

Les trois fonctionnalités bonus (meeting recorder, live WebRTC, watcher radio) ont émergé pendant la phase 4, quand l'infrastructure était suffisamment stable pour permettre d'ajouter des features sans casser l'existant.

6. Architecture d'ensemble

Le système est composé de 14 conteneurs Docker orchestrés par un unique fichier `docker-compose.yml`. Cette configuration me permet de tout déployer en une commande, avec les dépendances correctement ordonnées, les healthchecks configurés et les volumes de persistance nommés.

À l'entrée, un serveur Nginx écoute sur le port 443. C'est le seul port exposé publiquement. Il termine le TLS avec des certificats Let's Encrypt renouvelés automatiquement par certbot, puis fait un reverse proxy vers le frontend Next.js et l'API gateway. Tous les autres conteneurs sont bindés sur `127.0.0.1` et invisibles depuis l'extérieur.

Le frontend Next.js gère l'interface utilisateur. Il expose huit pages : la page d'accueil avec upload et micro, les pages d'authentification, la page meeting recorder, la page live WebRTC, et le dashboard admin. Toutes les communications avec le backend passent par la gateway.

La gateway est un service FastAPI qui joue trois rôles : elle authentifie les requêtes via JWT, elle expose l'API d'administration, et elle sert de proxy pour la création des tokens éphémères OpenAI Realtime utilisés par le live WebRTC. Elle communique en interne avec le pipeline via le réseau Docker.

Le pipeline est le cœur métier. C'est un service FastAPI qui embarque un orchestrateur LangChain LCEL. À chaque requête /process, il enchaîne trois étapes : appel au service STT pour transcrire l'audio, appel au service LLM pour traduire le texte, appel au service TTS pour synthétiser la voix. Chaque étape est un Runnable LangChain composable, ce qui rend la chaîne testable et instrumentable.

Les trois services d'inférence sont indépendants les uns des autres. Le STT utilise Faster-Whisper en version large-v3. Le LLM passe par LiteLLM qui route vers Groq, OpenAI ou Anthropic selon la configuration. Le TTS route vers Voxtral (Mistral) pour les langues majeures ou MMS-TTS (Meta) pour l'ukrainien et l'italien. Cette séparation permet de faire évoluer un composant sans toucher les autres.

En parallèle du pipeline synchrone, le service Watcher tourne en continu. Il a sa propre instance Whisper embarquée, ce qui lui évite un appel HTTP au service STT dédié à chaque cycle. Il appelle directement le service LLM sans passer par le pipeline (il n'a pas besoin de TTS puisqu'il ne produit que du texte structuré).

Côté outillage, MLflow tourne dans son propre conteneur pour le tracking d'expériences et le registre de modèles. Langfuse est utilisé en version cloud pour éviter d'héberger encore un service. Prometheus scrape les métriques toutes les 15 secondes, Grafana affiche douze panels de dashboards versionnés en Git. Airflow tourne en trois conteneurs (scheduler, webserver, base Postgres) et exécute les deux DAGs de batch.

7. Les choix techniques que j'ai faits

Chaque outil retenu l'a été après comparaison avec des alternatives. Je détaille ici les choix les plus structurants.

Frontend

J'ai choisi Next.js 15 en mode output: standalone, ce qui produit une image Docker de moins de 250 Mo. Le rendu côté serveur est natif, le hot reload en développement est confortable, et l'écosystème React reste le standard de l'industrie. J'ai écarté Streamlit, qui était initialement suggéré par mon mentor : son UX est trop rigide pour un vrai produit, notamment

pour gérer un enregistrement micro avec waveform en direct ou une connexion WebRTC. J'ai aussi écarté Vue.js (écosystème plus petit) et Angular (surdimensionné pour un frontend de cette taille).

Backend

FastAPI pour les six microservices. Le support natif d'async/await me permet de gérer plusieurs requêtes simultanément sans bloquer. La validation Pydantic est intégrée, la documentation OpenAPI est générée automatiquement, et les performances sont environ trois fois supérieures à Flask sur des benchmarks standards. J'ai écarté Flask (synchrone, sans typeage), Django (monolithique, trop lourd pour du microservice) et Express.js en Node (m'aurait obligée à réécrire toute la logique ML en JavaScript, ce qui n'a aucun sens).

Orchestration du pipeline

LangChain LCEL, avec son opérateur | qui compose les étapes comme dans un shell Unix. Chaque étape est un Runnable typé et testable. Le tracing Langfuse est gratuit à condition d'initialiser un client. J'ai considéré LangGraph, mais mon flow est linéaire (STT puis LLM puis TTS), sans branchement, donc LangGraph serait surdimensionné. J'ai aussi considéré un simple code Python maison, mais je perdrais l'écosystème LangChain (retry, tracing, composition, testing).

Couche LLM

LiteLLM comme couche d'abstraction. C'est un proxy Python qui unifie l'API vers plus de cent providers avec le même format que l'API OpenAI Chat Completions. Il gère les tarifs intégrés, le calcul du coût, la gestion des erreurs.

La preuve concrète que ce choix est le bon : en août 2026, Groq a déprécié le modèle gpt-oss-20b que j'utilisais alors en production. La migration vers OpenAI GPT-4o mini a demandé exactement une modification : la valeur de la variable d'environnement LLM_MODEL. Zéro ligne de code touchée dans mes services. C'est exactement ce qu'on attend d'une bonne couche d'abstraction.

Le modèle par défaut en production est actuellement openai/gpt-4o-mini. Latence sous deux secondes, coût autour de 0,0005 dollar par traduction, qualité BLEU largement suffisante pour ce cas d'usage.

Reconnaissance vocale (STT)

Faster-Whisper en version large-v3. C'est une implémentation CTranslate2 optimisée qui tourne quatre fois plus vite que le Whisper Python vanille en CPU. Le modèle supporte 99 langues nativement, ce qui me sert aussi pour le mode Live où l'utilisateur peut parler dans une langue autre que le français.

J'ai écarté l'API Whisper d'OpenAI (0,006 dollar la minute, pas de contrôle sur la version), Google Speech-to-Text (0,024 dollar la minute, moins précis en français technique) et AssemblyAI (0,015 dollar la minute, cher à volume).

Synthèse vocale (TTS)

Ici j'ai fait un choix hybride selon la langue. Voxtral, le TTS de Mistral, pour le français, l'anglais, l'espagnol et l'allemand. C'est un modèle récent, de qualité proche d'ElevenLabs, avec des voix naturelles. Pour l'ukrainien et l'italien, je route vers MMS-TTS de Meta, qui couvre plus de mille langues, y compris des langues rares.

Ce routage par langue m'évite d'avoir à choisir entre qualité (Voxtral) et couverture (MMS). J'ai écarté OpenAI TTS (excellent mais 15 dollars par million de caractères et pas d'ukrainien), ElevenLabs (premium mais très cher, pas d'ukrainien avant fin 2026) et XTTS/Coqui (fiabilité inférieure, dépendances Python lourdes).

Conteneurisation

Docker Compose plutôt que Kubernetes. Sur un seul VPS avec quatorze services, Kubernetes serait ridiculement surdimensionné. La courbe d'apprentissage aurait pris six mois. Docker Compose me donne un fichier YAML, une commande `up --build`, des healthchecks natifs. J'ai aussi considéré Docker Swarm (abandonné par la communauté) et Nomad de Hashicorp (peu répandu, peu de tutoriels).

Reverse proxy

Nginx avec certbot pour Let's Encrypt. C'est mature, éprouvé depuis vingt ans, avec des benchmarks de référence. La configuration reste lisible. J'ai considéré Traefik (config dynamique via labels Docker, mais moins lisible pour un projet de cette taille), Caddy (HTTPS automatique mais écosystème plus petit) et Apache (verbose et moins performant en reverse proxy).

Authentification

JWT custom plutôt qu'un service tiers. Les tokens d'accès sont signés en HS256 avec une durée de vie de quinze minutes. Les tokens de refresh sont aléatoires (32 bytes), stockés hashés en SHA-256 en base, et rotatifs à chaque utilisation (l'ancien est révoqué). Les mots de passe sont hashés avec bcrypt. J'ai vingt-cinq tests unitaires qui couvrent ce module : roundtrip, expiration, tampering, mauvaise signature, mauvais algorithme.

J'ai écarté Auth0 (23 dollars par mois minimum, dépendance externe critique), Firebase Auth (lock-in Google) et Keycloak (excellent mais un giga de RAM à lui tout seul, surdimensionné).

Orchestration batch

Airflow 2.10, avec deux DAGs. Le premier, `nightly_golden_eval`, tourne tous les jours à 2 heures UTC. Il extrait le golden dataset, appelle le pipeline en mode batch, calcule les métriques BLEU et METEOR, les logue dans MLflow, et déclenche une alerte Slack si la qualité chute. Le second, `weekly_drift_check`, tourne le dimanche à 3 heures UTC et compare le BLEU actuel à une baseline glissante sur sept jours.

J'ai écarté Prefect (API plus moderne mais moins répandu en entreprise, difficile à valoriser sur un CV), Dagster (excellent modèle de données mais courbe d'apprentissage) et le simple cron Linux (pas de monitoring, pas de retries, pas de dépendances entre tâches).

MLflow

Utilisé pour trois rôles simultanés : tracking d'expériences (soixante runs de comparaison de configurations), registre de modèles avec tag `production_version` sur les versions retenues, et évaluation via `mlflow.evaluate()` qui offre nativement huit métriques (BLEU, ROUGE-1/2/L, exact match, toxicity, Flesch-Kincaid, ARI). J'active aussi le tracing distribué avec des décorateurs `@mlflow.trace` sur les trois steps du pipeline, ce qui me donne des spans hiérarchiques visibles dans l'UI.

J'ai considéré Weights and Biases (excellent mais payant après le quota gratuit) et ClearML (moins standardisé en France).

Prometheus et Grafana

Prometheus scrape le endpoint `/metrics` des six services FastAPI toutes les quinze secondes, via la bibliothèque `prometheus-fastapi-instrumentator`. Grafana affiche douze panels : requêtes par seconde, latence p50, p95, p99, taux d'erreur, coût cumulé LLM, événements watcher. Les dashboards sont versionnés en Git dans `monitoring/grafana/dashboards/`.

J'ai écarté Datadog et New Relic (SaaS payants qui deviennent chers rapidement) et la stack ELK (lourde, plusieurs gigas de RAM juste pour ElasticSearch).

Langfuse

Complémentaire à Prometheus, mais focalisé sur le versant métier LLM. Il capture chaque appel avec l'input, l'output, la latence, le coût, les tokens. La vue waterfall affiche les trois spans du pipeline (STT, LLM, TTS) dans l'ordre chronologique. Le versioning des prompts (v1.0, v1.1, v1.2) est fait de son côté. J'utilise la version cloud pour éviter d'héberger encore un service, mais la version self-hosted est disponible.

J'ai écarté LangSmith (couplé à LangChain, moins ouvert), Arize Phoenix (orienté entreprise) et Helicone (proxy HTTP intrusif).

Une note importante : Langfuse a publié sa v4 pendant mon développement, avec des breaking changes SDK. J'ai dû refactorer le client complet, migrer les kwargs `start_time` et `end_time` vers le dictionnaire `metadata`, et basculer sur `start_as_current_observation()` en context manager. Migration transparente pour l'utilisateur final, mais quelques heures de dev pour moi.

8. Ingénierie des données et évaluation

Le dataset golden compte trente audios de flash-infos radio 107.7 réels, capturés avec des conditions variées (audio propre, audio bruité, voix féminines et masculines, différentes zones géographiques). Chaque audio est accompagné d'une traduction humaine validée dans les cinq langues cibles. L'ensemble est stocké dans `data/golden/` et versionné en Git.

Pour la phase de sélection, j'ai construit un plan d'expérience à douze configurations. Deux modèles Whisper (small pour la rapidité, large-v3 pour la qualité), deux modèles LLM (Llama 3.1 8B et 70B, à l'époque via Groq), et trois versions de prompt (v1.0 basique, v1.1 pro traffic, v1.2 broadcast quality). Chaque configuration testée sur les cinq audios golden, ce qui donne soixante runs enregistrés dans MLflow.

Les métriques utilisées sont classiques en évaluation de traduction. BLEU mesure la similarité en n-grams avec la traduction humaine de référence. METEOR est une version pondérée qui prend en compte les synonymes. ROUGE-1, ROUGE-2 et ROUGE-L couvrent le recall. J'ai aussi mesuré le WER (Word Error Rate) sur la partie STT, la latence par étape via Prometheus, et le coût par requête via LiteLLM.

Les résultats sont intéressants. Le champion BLEU absolu est Llama 70B avec le prompt v1.1, à 51,3 points. Mais Llama 8B avec le même prompt monte à 46,2 points, pour un coût dix fois inférieur. J'ai donc retenu Llama 8B en production. Un gain de cinq points BLEU ne justifie pas de multiplier la facture par dix, surtout que le contenu à traduire (des flash-infos courts et structurés) n'a pas besoin de la finesse du 70B.

Suite à la dépréciation de Llama 3.1 8B chez Groq en août 2026, j'ai migré vers OpenAI GPT-4o mini. La qualité est comparable, la latence est légèrement meilleure, et le coût reste très raisonnable (environ 0,0005 dollar par traduction).

L'évaluation continue est automatisée par le DAG Airflow `nightly_golden_eval`. Il tourne toutes les nuits, ré-évalue les mêmes trente audios avec la configuration de production actuelle, et logue les nouveaux scores dans MLflow. Si une régression est détectée (baisse de BLEU au-delà d'un seuil), une alerte Slack est envoyée. Le DAG `weekly_drift_check` complète ce dispositif en comparant la baseline glissante sur sept jours, ce qui capture les dérives lentes que le `nightly` ne verrait pas.

9. Sécurité

La sécurité est traitée en trois couches, selon le principe de défense en profondeur.

La couche réseau isole tout ce qui peut l'être. Les quatorze conteneurs Docker sont bindés sur 127.0.0.1 et invisibles depuis Internet. Seul Nginx expose le port 443 en HTTPS. Les certificats Let's Encrypt sont renouvelés automatiquement par certbot. Cette isolation garantit qu'un attaquant qui aurait un exploit sur MLflow ou sur Grafana ne peut simplement pas les atteindre depuis l'extérieur.

La couche authentification applicative repose sur du JWT custom. Les tokens d'accès sont signés en HS256 avec une durée de vie courte de quinze minutes. Les tokens de refresh sont plus longs (sept jours) mais hashés en SHA-256 en base, et surtout rotatifs (l'ancien est révoqué à chaque utilisation). Les mots de passe sont hashés avec bcrypt en salt automatique. Ce module est couvert par vingt-cinq tests unitaires qui vérifient le roundtrip, l'expiration, le tampering, les mauvaises signatures et les mauvais algorithmes. J'ai choisi cette approche custom plutôt qu'Auth0 ou Firebase pour deux raisons : le contrôle total, et l'absence de dépendance externe critique. Cent lignes de code suffisent, et je peux tout auditer.

La couche LLM est celle qui m'a demandé le plus d'attention, parce qu'elle est la plus spécifique et la moins standard. La menace principale est l'injection de prompt, cataloguée OWASP LLM01. Un audio malicieux pourrait contenir des instructions cachées comme « ignore previous instructions » ou « reveal your system prompt ». J'ai mis en place trois protections. D'abord un pre-check regex bilingue (français et anglais) qui détecte les patterns d'injection connus et bloque la requête avant même qu'elle n'atteigne le LLM. Ensuite un sandboxing qui échappe les balises XML dans le texte utilisateur. Enfin un post-check qui vérifie le ratio de longueur input/output (une hallucination produit souvent un output démesuré à partir d'un input court) et détecte les prompt leaks classiques (« I am an AI », « my instructions say »). Ce module est couvert par vingt-quatre tests unitaires.

10. Observabilité

J'ai adopté une approche à trois niveaux, complémentaires les uns des autres.

Le niveau infrastructure est couvert par Prometheus et Grafana. Prometheus scrape les métriques HTTP standards (requêtes par seconde, latence, erreurs 4xx et 5xx) sur les six services FastAPI, plus les métriques système (RAM, CPU). Les dashboards Grafana affichent douze panels organisés par service et par percentile de latence. Cette couche répond aux questions ops : est-ce que le service est up, combien de requêtes traite-t-il, quelle est la latence p95.

Le niveau applicatif est couvert par le tracing MLflow. J'ai décoré les trois steps du pipeline (`_stt_step`, `_llm_step`, `_tts_step`) avec `@mlflow.trace`. Chaque appel au pipeline produit un span parent avec trois sous-spans hiérarchisés. Je vois dans l'UI MLflow quel step prend combien de temps, quels arguments ont été passés, quelle réponse a été reçue. C'est très utile pour déboguer un ralentissement ou une erreur intermittente.

Le niveau métier LLM est couvert par Langfuse. Cet outil est spécialisé dans l'observabilité des applications LLM. Il capture chaque appel avec l'input, l'output, la latence, le coût, les tokens consommés. La vue waterfall affiche les spans dans l'ordre chronologique. Les prompts sont versionnés (v1.0, v1.1, v1.2), ce qui me permet de savoir a posteriori quelle version a été utilisée pour telle requête. Cette couche répond aux questions métier : combien coûte une traduction, quelle version de prompt donne les meilleurs résultats, quelles requêtes prennent le plus de tokens.

Les trois couches se recoupent partiellement mais couvrent des besoins différents. Prometheus me dit qu'un service est lent, MLflow me dit à quelle étape le ralentissement se produit, Langfuse me dit combien coûte cette lenteur.

11. Déploiement et intégration continue

La production tourne sur un VPS Hetzner sous Ubuntu Server. La machine dispose de 62 gigaoctets de RAM, sans GPU dédié. Le domaine `traduction-audio.fr` est géré par OVHcloud, avec le HTTPS assuré par Let's Encrypt et le renouvellement automatique via certbot.

L'intégration continue tourne sur GitHub Actions. À chaque push sur la branche main, un workflow lance la suite pytest complète en environ cinq secondes. Le rapport JUnit est uploadé en artifact. Un échec bloque le merge de la branche. Le workflow est défini dans `.github/workflows/ci.yml`.

Le déploiement continu est en place via un second workflow `.github/workflows/deploy.yml`, qui se déclenche automatiquement après un CI réussi sur main. Il utilise SSH pour se connecter au serveur, fait un git pull, exécute le script `scripts/deploy.sh` qui rebuild les images sans cache et force-recreate les containers, puis vérifie les healthchecks post-déploiement. Un déclenchement manuel est aussi possible via l'interface GitHub Actions, ce qui est utile pour redéploier un service spécifique.

Les tests sont organisés en trois catégories. La suite unit contient 172 tests couvrant les modules critiques : authentification JWT, chunking de long audios, routing TTS par langue, alias de modèles, prompt-guard, résistance aux injections. La suite intégration contient 23 tests qui couvrent les flows bout-en-bout : login, pipeline complet, appels HTTP entre services. Une suite e2e est prévue en phase 2, avec Playwright pour tester l'interface Next.js.

12. Résultats et chiffres

Sur la qualité de traduction, le score BLEU champion est de 46,2 avec la configuration de production (Llama 8B v1.1). Le score METEOR moyen est de 0,72. Le WER de Whisper large-v3 sur des audios propres est autour de 7%.

Sur la performance, la latence end-to-end oscille entre deux et trois secondes selon la longueur de l'audio. Le détail est le suivant : environ 800 millisecondes pour le STT (Whisper large-v3 en CPU), 1,2 seconde pour le LLM (GPT-4o mini via LiteLLM), 600 millisecondes pour le TTS (Voxtral). Le mode Live WebRTC est nettement plus rapide avec 500 millisecondes de latence first-byte, puisque OpenAI Realtime fait tout en interne sans passer par mes services.

Sur le coût, une traduction upload revient à moins de 0,001 dollar en production. Une session live coûte environ 0,30 dollar par minute (OpenAI Realtime est plus cher, environ dix fois le coût d'un pipeline standard). Le coût mensuel d'infrastructure hermes est d'environ 30 euros par mois.

Sur les volumes, le projet compte 60 runs MLflow d'expérimentation, 195 tests automatisés (172 unit et 23 integration), 14 conteneurs Docker en production, 24 routes API sur la gateway, 8 pages frontend, 6 langues supportées (français source, plus anglais, espagnol, allemand, italien, ukrainien en cible), et environ 18 outils MLOps distincts intégrés.

13. Difficultés rencontrées

Je détaille ici les problèmes les plus significatifs que j'ai dû résoudre, parce qu'ils illustrent bien les défis d'un projet LLMOps réel.

La migration de Langfuse v2 vers v4 a été mon plus gros refactor. La v4 a introduit un nouveau modèle avec context managers (`start_as_current_observation()`), et les arguments `start_time` et `end_time` sont passés dans le dictionnaire metadata au lieu de kwargs. Toute mon instrumentation du pipeline a dû être réécrite. Une bonne demi-journée de travail, mais la migration est transparente pour l'utilisateur.

La dépréciation successive par Groq du modèle `llama-3.1-8b-instant` puis de `gpt-oss-20b` en été 2026 a provoqué des erreurs 500 en cascade dans le pipeline. J'ai résolu ça avec un pattern de model aliases côté serveur : un dictionnaire qui redirige transparentement les anciens noms de modèles vers OpenAI GPT-4o mini. Aucun changement côté client. Ce cas concret a validé rétroactivement mon choix architectural de LiteLLM comme couche d'abstraction : quand un provider dégrade son offre, la migration reste triviale.

L'enregistreur de meeting a eu un bug subtil : les chunks 2 et suivants revenaient transcrits comme vides. La cause était dans la logique du MediaRecorder du navigateur : appeler `start(30_000)` avec un timeslice génère des fragments WebM sans header, et Whisper décode correctement le premier fragment mais échoue silencieusement sur les suivants. La solution

a été de passer sur un pattern stop+restart du MediaRecorder : chaque cycle stop produit un fichier WebM complet avec header, et je redémarre immédiatement un nouveau cycle. Un flag `stoppingRef` distingue les arrêts volontaires des rotations planifiées, pour ne pas redémarrer indéfiniment.

Le mode Live WebRTC a eu un problème physique de feedback loop. Sans casque, le speaker rejoue la traduction dans le micro, le modèle OpenAI se re-traduit lui-même, et on se retrouve dans une boucle infinie de délires (« Wait a minute » qui devient « How much is it » qui devient « Really? » qui devient « OK »). La solution logicielle a été de mute automatiquement le track micro dès que le modèle commence à parler (event `response.audio.delta`), puis de le unmute à la fin (`response.done`). Le feedback loop devient physiquement impossible même sans casque.

Le cookie de session Next.js avait un TTL de quinze minutes alors que le refresh JWT dure sept jours. Résultat : au bout de quinze minutes d'inactivité, l'utilisateur était redirigé vers `/login` alors qu'il pouvait encore rafraîchir son token silencieusement. J'ai aligné la durée du cookie sur celle du refresh (sept jours). Le cookie ne sert que d'indicateur « il y a une session en cours » pour le middleware Next.js ; la vraie validation JWT reste faite par la gateway sur chaque appel API.

Whisper avait tendance à halluciner sur les accents. Sans hint de langue explicite, un français avec un accent (russe, ukrainien, italien) était parfois transcrit dans la mauvaise langue, ce qui rendait ensuite la traduction absurde. La solution a été de passer explicitement `language: "fr"` dans la config transcription, et d'ajouter dans le mode Live un sélecteur « Je parle en » qui permet à l'utilisateur de forcer une autre langue source.

14. Perspectives

Le projet est livré en phase 1. La phase 2 comporte plusieurs chantiers.

Côté technique, j'aimerais ajouter MinIO comme stockage S3-like pour préparer un éventuel scaling multi-instance. Ragas apporterait des métriques d'évaluation LLM plus avancées, même si le use case traduction (par opposition au RAG) le rend moins critique. Evidently permettrait une détection de drift au niveau des features. Une pyannote.audio pourrait ajouter de la diarization (identification du locuteur) au meeting recorder, pour un compte-rendu nominatif du type « Alice a dit » et « Bob a proposé ». Enfin, je voudrais étendre le dataset golden de trente à cent audios, ce qui donnerait plus de robustesse aux évaluations nightly.

Côté produit, deux débouchés se dessinent. Un débouché B2B via un partenariat avec un opérateur autoroutier (Vinci, APRR, ATMB) pour intégrer traduction-audio dans leurs applications mobiles usagers. Un débouché B2C via une offre freemium sur le meeting recorder, qui a un vrai potentiel de monétisation SaaS. Il y a aussi la question de l'extension linguistique : passer de six à quinze langues européennes est faisable puisque Whisper les gère nativement, il suffit d'ajouter les prompts et de vérifier les modèles TTS disponibles.

Sur le plan des compétences, ce projet me valide un ensemble transférable : Python asynchrone avec FastAPI, TypeScript et React avec Next.js 15, Docker Compose multi-service, l'écosystème LLMOps complet (LangChain, LiteLLM, MLflow, Langfuse), les fondamentaux DevOps (Nginx, Let's Encrypt, GitHub Actions, VPS deployment) et la sécurité applicative (JWT, bcrypt, prompt-guard).

15. Conclusion

Ce projet démontre qu'il est possible aujourd'hui de construire une plateforme LLMOps complète, sécurisée et déployée en production réelle, en s'appuyant sur des modèles pré-entraînés et un écosystème d'outils matures. L'enjeu n'est pas la modélisation, mais l'orchestration.

Les quatre piliers MLOps classiques (orchestration, inférence, monitoring, stockage) sont traités chacun comme un citoyen de première classe, avec un outil dédié et une architecture pensée pour la maintenabilité. Les cinq piliers étendus (avec l'ajout du versioning et de la sécurité) sont couverts par environ dix-huit outils intégrés.

Le projet livre plus que ce qui était prévu au départ. Les trois fonctionnalités bonus (meeting recorder, live speech-to-speech, watcher radio) sont apparues pendant le développement et ouvrent des débouchés concrets. Le coût par traduction en dessous du millième de dollar, la latence end-to-end de deux à trois secondes, la qualité BLEU de 46,2 sur le golden dataset, et l'ensemble des chiffres du projet (14 conteneurs, 195 tests, 60 runs MLflow, une infrastructure déployée sur un vrai domaine avec HTTPS) attestent d'un niveau de maturité applicable dans un contexte professionnel.

Il reste bien sûr des choses à améliorer, dont plusieurs sont listées dans les perspectives. Mais la fondation est solide et le socle technique est prêt à accueillir une phase 2 ambitieuse.

16. Annexes techniques

Structure du repository

```
translate-audio-NLP-Ai/
├── backend/services/      # 6 microservices FastAPI
│   ├── gateway/          # Auth JWT + Admin API
│   └── pipeline/          # Orchestrateur LangChain
├── LCEL
│   ├── stt/              # Faster-Whisper large-v3
│   ├── llm/              # LiteLLM multi-provider
│   ├── tts/              # Voxtral + MMS-TTS
│   └── watcher/          # Poll radio 107.7
├── frontend/             # Next.js 15 standalone
├── airflow/dags/          # 2 DAGs Python
└── monitoring/           # Prometheus + Grafana
```

configs	
— scripts/	# 10 scripts métier
— tests/	# 195 tests (unit +
integration)	
— docs/	# ARCHITECTURE + RUNBOOK +
rapport	
— data/golden/	# Audios de référence
— docker-compose.yml	# 14 services orchestrés

Documentation associée

Le repository contient plusieurs documents complémentaires. Le README.md donne un quickstart et un overview technique. Le docs/ARCHITECTURE.md détaille la vue technique. Le docs/RUNBOOK.md décrit les procédures opérationnelles. Le docs/architecture-schema.txt fournit un schéma ASCII complet en fichier texte. Le docs/soutenance.html est le support de soutenance interactif à trente slides.

Commandes utiles

Pour un setup local complet :

```
git clone git@github.com:TatianaT13/translate-audio-NLP-
    Ai.git
cd translate-audio-NLP-Ai
cp .env.example .env
# renseigner les clés API dans .env
docker compose up -d --build
```

Pour lancer les tests :

```
pytest tests/unit/          # 172 tests, environ 5 secondes
pytest tests/integration/    # 23 tests, environ 30 secondes
```

Pour déployer en production :

```
./scripts/deploy.sh          # rebuild tous les services
./scripts/deploy.sh frontend # rebuild uniquement le
                             frontend
```

Pour lancer une évaluation batch manuelle :

```
python scripts/eval_golden.py
python scripts/import_metrics_to_langfuse.py
```

Contacts

Auteur : Tetyana Tarasenko Mentor : Sébastien (DataScientest) Providers
LLM : Groq, OpenAI, Anthropic Hébergement : Hetzner (VPS) et
OVHcloud (DNS)

Rapport rédigé pour la soutenance du 3 septembre 2026.